

Laboratorio #4 - Fundamentos de RL y Gymnasium

CC3092 - Deep Learning y Sistemas Inteligentes

Repositorio: github.com/jlope384/Laboratorio-4-Reinforcement-Learning-101

1. Fundamentos del aprendizaje por refuerzo

Que es RL y diferencias con supervisado/no supervisado

El aprendizaje por refuerzo es la rama del aprendizaje automatico donde un agente aprende probando cosas dentro de un entorno y viendo que recompensa le da cada decision, en vez de que le digan de entrada cual es la respuesta correcta. A diferencia del aprendizaje supervisado, aca no hay un dato etiquetado que diga cual era la mejor accion, solo un numero que ademas suele llegar tarde y depende de toda una secuencia de decisiones. Y a diferencia del no supervisado, si hay una senal externa (la recompensa), solo que esta indica que tan bien salio la jugada, no cual hubiera sido la jugada optima.

Componentes, MDP, $V(s)/Q(s,a)$ y ecuacion de Bellman

Un problema de RL se arma con agente, entorno, estados, acciones, recompensas y una politica que decide que hacer en cada estado. Formalmente se modela como un Proceso de Decision de Markov: la tupla (S, A, P, R, γ) , con el conjunto de estados y acciones, una funcion de transicion $P(s'|s,a)$, una de recompensa R y un factor de descuento γ que pesa las recompensas futuras frente a las inmediatas. $V(s)$ es el retorno esperado desde el estado s siguiendo una politica dada; $Q(s,a)$ es lo mismo pero fijando tambien la primera accion. La politica optima elige en cada estado la accion con mayor Q , $\pi^*(s) = \operatorname{argmax}_a Q^*(s,a)$. La ecuacion de Bellman conecta el valor de un estado con el de los estados siguientes: el valor de estar aca es la recompensa inmediata mas el valor descontado de a donde se llega despues, y esa relacion recursiva es lo que permite calcular V y Q de forma iterativa (programacion dinamica, Monte Carlo, TD-learning).

Exploracion vs. explotacion, tipos de tareas/metodos, y Q-Learning

El dilema exploracion-explotacion aparece porque el agente tiene que elegir entre la accion que segun lo que sabe le conviene mas (explotar) o probar algo distinto para conseguir mas informacion (explorar). Dos formas comunes de manejarlo: epsilon-greedy, donde con probabilidad epsilon se elige una accion al azar, y softmax/Boltzmann, donde se elige de forma probabilistica ponderando por los valores Q . Una tarea episodica termina en un estado terminal; una continua no tiene fin natural. Un metodo on-policy aprende sobre la misma politica que usa para actuar (SARSA); uno off-policy puede aprender sobre una politica distinta de la que usa para explorar (Q-Learning). Model-based es cuando el agente usa un modelo de la dinamica del entorno para planificar; model-free es cuando aprende directo de la experiencia sin ese modelo. La actualizacion de Q-Learning es $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$: α controla cuanto se mueve la estimacion hacia lo que se acaba de observar, y γ pesa el valor futuro estimado; usar $\max_{a'}$ en vez de la accion realmente tomada es justo lo que lo hace off-policy.

2. La libreria Gymnasium

Gymnasium (Farama Foundation) es la continuacion mantenida de OpenAI Gym; da una API estandar (reset, step, render, close) para definir entornos de RL, resolviendo el problema de que antes cada quien implementaba sus entornos con una interfaz distinta. reset() reinicia el entorno y devuelve (observation, info). step(action) avanza un paso y devuelve (observation, reward, terminated, truncated, info): terminated es True cuando el episodio termino por una razon propia del MDP, y truncated cuando se corto por algo externo, tipicamente un limite de pasos. render() genera una representacion visual del estado; close() libera los recursos que el entorno haya usado.

Los Spaces definen que valores son validos. Discrete(n) es un conjunto finito de n valores categoricos (las 2 acciones de CartPole, por ejemplo); Box(low, high, shape) es un vector continuo n -dimensional (la observacion de 4 valores de CartPole); MultiDiscrete([$n_1, n_2, \dots$]) son varias variables discretas independientes combinadas en una sola accion compuesta.

Cuatro entornos del catalogo: CartPole-v1 (Classic Control), equilibrar un poste sobre un carro, obs. Box(4,) [pos., vel., angulo, vel. angular], accion Discrete(2). MountainCar-v0 (Classic Control), llevar un auto a la cima usando impulso, obs. Box(2,) [pos., vel.], accion Discrete(3). FrozenLake-v1 (Toy Text), cruzar una grilla congelada hasta la meta sin caer en un hoyo, obs. Discrete(16), accion Discrete(4). LunarLander-v3 (Box2D), aterrizar una nave entre dos banderas, obs. Box(8,) [pos., vel., angulo, vel. angular, 2 contactos], accion Discrete(4).

Un wrapper envuelve un Env para cambiarle o agregarle comportamiento sin tocar su codigo interno, manteniendo la misma interfaz. Por ejemplo TimeLimit limita el numero de pasos por episodio y marca truncated=True al llegar al limite; RecordVideo graba episodios en video usando render_mode='rgb_array'; y los wrappers de normalizacion reescalan la observacion (p. ej. a media 0 / varianza 1) para estabilizar el entrenamiento.

3. Modulo de prueba: resultados

Se utilizo gymnasium version 1.3.0. Para CartPole-v1, observation_space es Box(4,) con limites [-4.8, -inf, -0.419, -inf] a [4.8, inf, 0.419, inf], y action_space es Discrete(2).

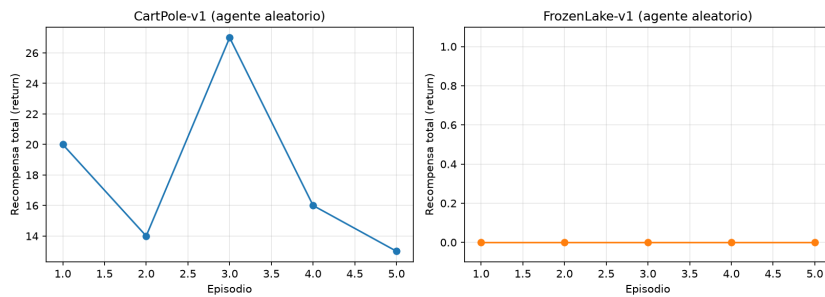


Figura 1. Recompensa por episodio del agente aleatorio en CartPole-v1 (izq.) y FrozenLake-v1 (der.), 5 episodios cada uno.

Agente aleatorio, 5 episodios: en CartPole-v1 el return promedio fue 18.00 (steps promedio 18.0), variando entre 13 y 27 por episodio. En FrozenLake-v1 (grilla 4x4, is_slippery=True) el return promedio fue 0.00, con 0 de 5 episodios exitosos (el agente nunca alcanzo la meta).

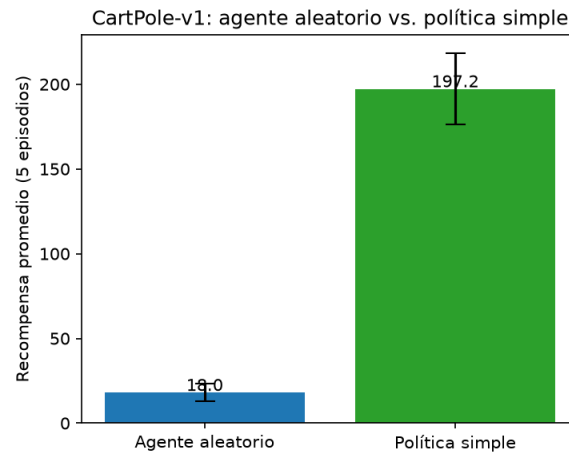


Figura 2. Recompensa promedio (+/- desv. estandar) en CartPole-v1: agente aleatorio vs. política simple (5 episodios c/u).

Política simple para CartPole-v1 ('si la velocidad angular del poste es positiva, empujar a la derecha; si es negativa, empujar a la izquierda'), 5 episodios: return promedio 197.20 (+/- 21.06), muy por encima del agente aleatorio (return promedio 18.00 +/- 5.10), una mejora de +179.20 puntos en promedio.

4. Discusion y analisis

- CartPole vs. FrozenLake: en CartPole la recompensa es densa (+1 por cada paso sobrevivido), asi que cualquier secuencia de acciones al azar ya acumula algo antes de que el poste se caiga (return promedio 18.00). En FrozenLake la recompensa solo aparece al llegar a la meta y el piso es resbaladizo, asi que el agente aleatorio no logro exito ni una vez en los 5 episodios (return promedio 0.00). La diferencia esta en eso: que tan densa es la recompensa y que tan estocastico es el entorno determinan si actuar al azar sirve para algo.
- La politica simple si le gana por mucho al agente aleatorio (197.20 contra 18.00 de return promedio). Con solo una regla basada en la velocidad angular del poste, sin entrenar nada, ya se consigue una mejora enorme. Eso si, la regla usa una sola variable del estado; una politica aprendida via Q-Learning probablemente la superaria aun mas al aprovechar todas las variables, ademas de poder adaptarse a otros entornos sin tener que rediseñarla a mano.
- Incluso con una politica que funciona bien hace falta explorar porque puede no estar calibrada en los estados que casi nunca visita; sin probar otras acciones ahi, nunca se sabria si hay algo mejor. Ese es el dilema de exploracion vs. explotacion: quedarse solo con lo conocido da resultados decentes de inmediato pero arriesga quedar atrapado en un optimo local que no es realmente el mejor.
- LunarLander-v3 me parece el mas interesante para un laboratorio futuro: tiene una observacion continua de 8 dimensiones y una accion discreta de 4, un salto de complejidad claro respecto a CartPole, y su recompensa combina varios factores (combustible, que tan suave fue el aterrizaje) lo que da pie a probar reward shaping y metodos como Deep Q-Networks, mas cercanos al enfoque del curso.